Design and application of a Graph Library in Julia for Chemical Space Networks

BENZIADA Nadine | MAIO André

MSc. Chemoinformatics / Chemoinformatics IV

Current Year: 2026

Professor:
MARCOU Gilles

Contents

1 Introduction **2**

2 Project Objectives **2**

3 Quick Start Instructions **3**

4 User Manual **4**

4.1 Input Formats 4

4.1.1 Adjacency-list format (`.graph`) 4

4.1.2 Edge-list format (`.txt` / `.csv`) 4

4.2 Graph Loading 4

4.3 Analysis Tools 5

4.4 Visualization Output 5

5 Algorithm **5**

5.1 Graph Loading Algorithm 6

5.2 Connected Components Detection 7

5.3 Graph Normalization for Isomorphism 7

6 Conclusions and Perspectives **8**

1 Introduction

Graphs constitute a fundamental mathematical structure for modeling relational data and are widely used in scientific domains where complex interactions must be analyzed. In chemoinformatics, graph-based representations are particularly well suited for describing chemical space networks, in which nodes correspond to molecules and edges encode similarity or structural relationships.

The objective of this project is to design and implement a graph-based analysis software using the Julia programming language. The project focuses on the complete software workflow, including graph loading, algorithmic analysis, and result interpretation, while maintaining clarity, robustness, and reproducibility.[1] Particular attention is given to the integration of classical graph algorithms that are relevant to chemoinformatics applications.

The main challenges addressed include supporting multiple graph input formats, orchestrating fundamental graph algorithms within a unified execution model, and providing an intuitive graphical interface requiring minimal user interaction. The resulting software enables both quantitative analysis and visual exploration of chemical space networks, offering a practical demonstration of how graph theory concepts can be applied in a chemoinformatics context.

2 Project Objectives

The objectives of this project are to:

- Design and implement a graph analysis framework in Julia, with a clear and explicit data storage model.
- Propose an efficient graph representation suitable for traversal and analysis, while discussing the associated design trade-offs.
- Support graph construction, loading, and persistence through simple, human-readable input formats.
- Implement core graph algorithms such as graph traversal, connectivity analysis, and structural comparison with minimal user interaction.
- Demonstrate the applicability of graph-based methods to chemoinformatics through the analysis and visualization of chemical space networks.

3 Quick Start Instructions

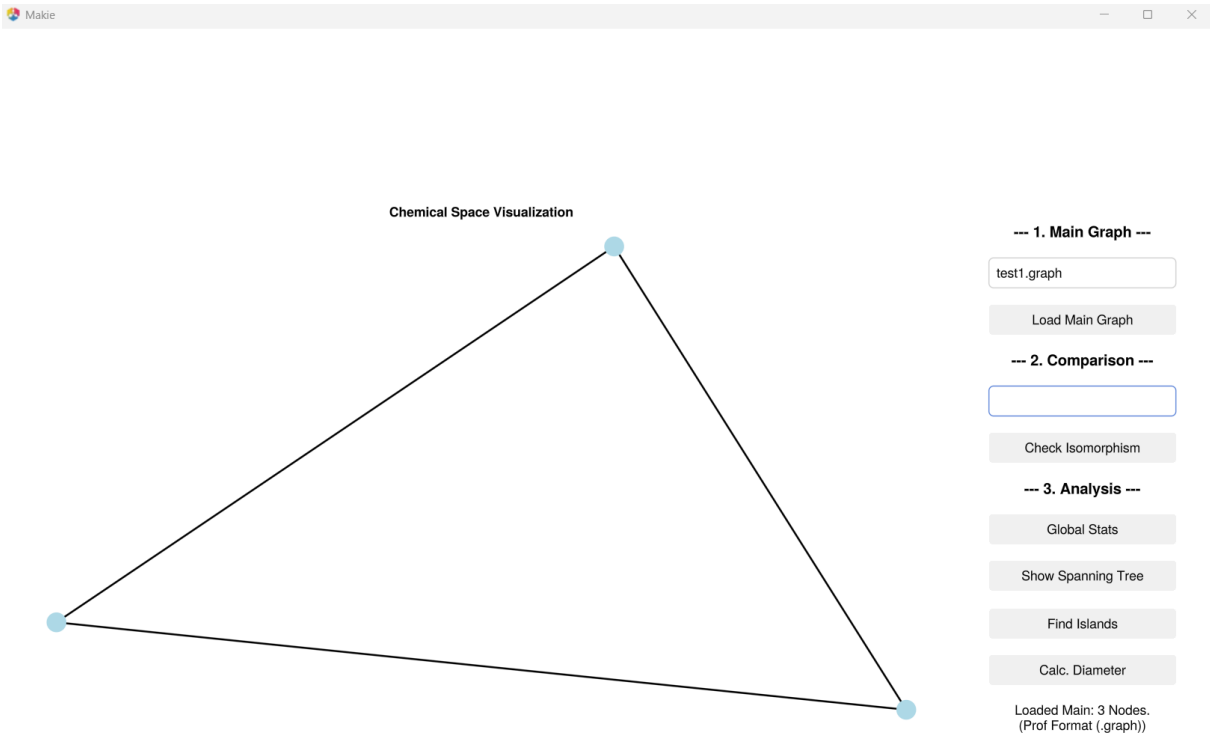

Figure 1: Example output of the JuliaGraphViz after loading a graph. The main visualization displays the graph structure, while the control panel provides access to analysis tools and textual feedback.

Prerequisites

- Julia version 1.6 or higher
- Required packages: `GLMakie`, `GraphMakie`, `Graphs`, `NetworkLayout`

Execution Steps

1. Place `final_project.jl` and a graph file (e.g. `test1.graph`) in the same directory.
2. Open a terminal in this directory.
3. Run the program:

   ```
   julia final_project.jl
   ```

4. A graphical window opens automatically.
5. Enter `test1.graph` and click **Load Main Graph**.

Expected Output

If the software is functioning correctly, the graph is displayed and the status panel reports the number of nodes, edges, and detected format. Clicking **Global Stats** displays structural metrics such as density and average degree.

4 User Manual

4.1 Input Formats

The software supports two text-based graph formats.

4.1.1 Adjacency-list format (`.graph`)

This format represents graphs using adjacency lists and may include optional comments:

```
# Header: Nodes,Edges
3,3
1 : 2,3
2 : 3
3 :
```

4.1.2 Edge-list format (`.txt` / `.csv`)

Each line represents an undirected edge between two nodes:

```
1,2
2,3
3,1
```

In both formats, node indices start at 1. Graphs are treated as undirected and unweighted.

4.2 Graph Loading

To load a graph, the user must:

- Enter the filename in the input text field.
- Press **ENTER** to validate the input.
- Click **Load Main Graph**.

The software automatically detects the input format and parses the file accordingly.

4.3 Analysis Tools

The graphical interface provides the following analysis tools:

Button	Function	Interpretation
Global Stats	Computes global graph statistics (density, average degree)	Provides a quantitative overview of network sparsity
Show Spanning Tree	Computes and highlights the minimum spanning tree (or forest)	Reveals the structural backbone of the graph
Find Islands	Detects connected components in the graph	Identifies clusters of related nodes
Calc. Diameter	Computes the graph diameter when applicable	Estimates the maximum separation between nodes
Check Isomorphism	Tests structural equivalence between two graphs	Determines whether two graphs share the same topology

4.4 Visualization Output

The visualization layer displays analysis results as follows:

- Nodes are displayed as colored circles.
- Connected components are visualized using distinct colors.
- Minimum spanning tree edges are highlighted in red.
- Redundant edges are shown in light gray.

Important note: To ensure consistent visualization results, the graph should be reloaded between different analyses in order to reset the visualization state.

5 Algorithm

This section describes the most relevant algorithms implemented in the software. The overall execution workflow is summarized by the logigram shown in Figure 2, which illustrates the sequence of operations from graph loading to result visualization. The internal graph representation and traversal operations rely on the Graphs.jl library, which provides efficient and well-tested graph abstractions and algorithms [2].

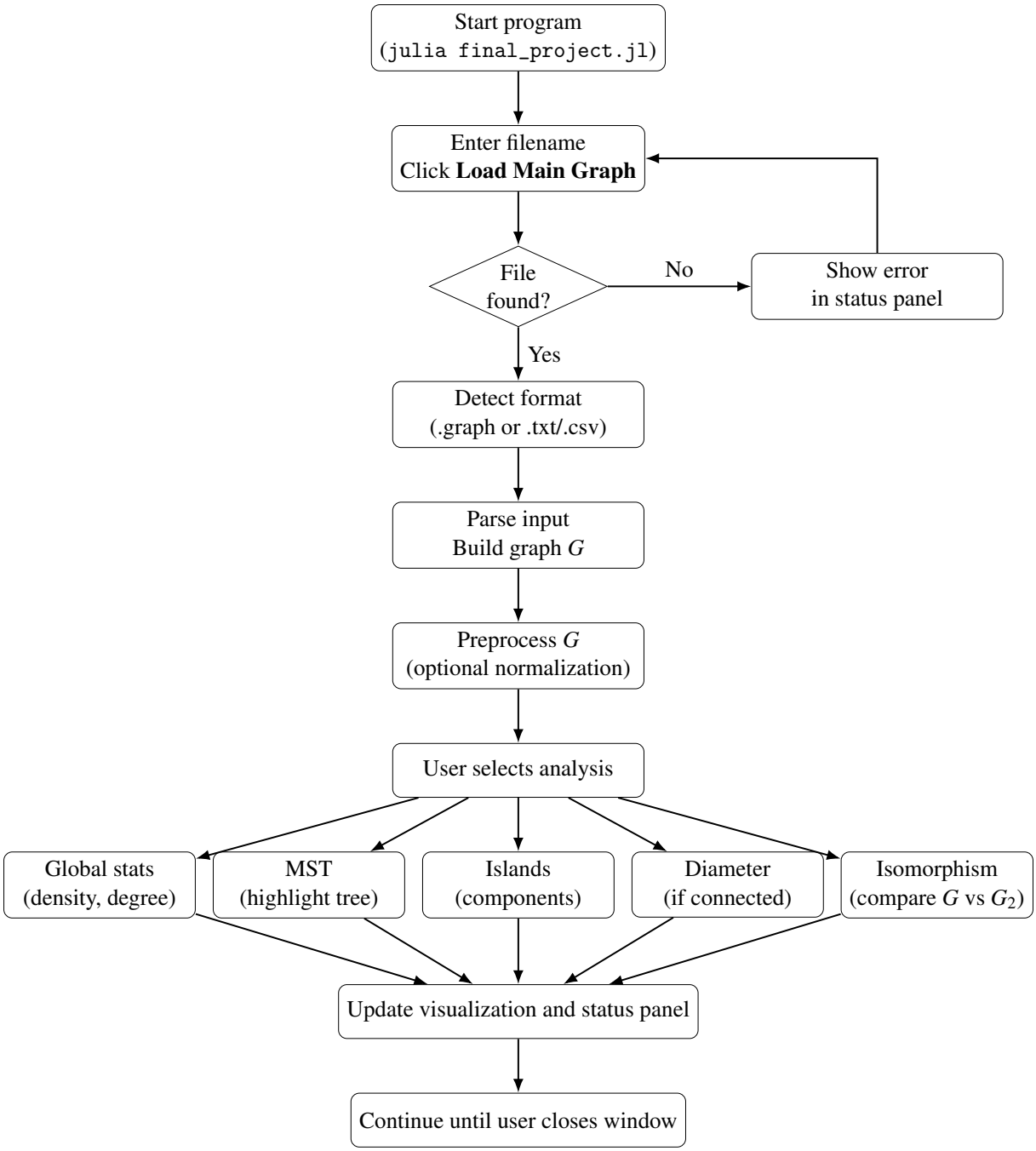

Figure 2: Logigram of the software workflow. After file loading and format detection, the graph is parsed and optionally normalized. The user then selects one of several graph analyses (global statistics, minimum spanning tree, connected components, diameter, or isomorphism), whose results are displayed through textual feedback and interactive visualization in the GUI.

5.1 Graph Loading Algorithm

Purpose. Parse a text-based graph file and construct the corresponding internal graph representation.

Pseudocode.

```
Algorithm LoadGraph(file):
    Read all lines from file
```

```
    Remove comments and empty lines
    Detect format based on content
    Initialize graph with correct number of nodes
    For each line describing edges:
        Add corresponding edges to graph
    Return constructed graph
```

This algorithm runs in linear time with respect to the number of nodes and edges.

5.2 Connected Components Detection

Connected components are computed using a breadth-first search (BFS) traversal. This allows the identification of groups of nodes that are mutually reachable.

Pseudocode.

```
Algorithm ConnectedComponents(G):
    Mark all nodes as unvisited
    For each node v in G:
        If v is unvisited:
            Run BFS from v
            Mark all reachable nodes as one component
```

Complexity. The time complexity is $O(n+m)$, where n is the number of nodes and m is the number of edges.

5.3 Graph Normalization for Isomorphism

Before graph isomorphism testing, isolated nodes are removed and the remaining nodes are reindexed. This normalization step ensures that structurally identical graphs with different node labels or indexing schemes can be compared meaningfully. The normalization is applied only when required, in particular for isomorphism testing.

Algorithm 1 Graph Isomorphism Workflow

Load graphs G_1 and G_2
Normalize G_1 and G_2 by removing isolated vertices
if $|V(G_1)| \neq |V(G_2)|$ **then**
    return Not isomorphic
end if
Apply structural isomorphism test to G_1 and G_2
if topologies match **then**
    return Isomorphic
else
    return Not isomorphic
end if

6 Conclusions and Perspectives

Observations

Using the software highlighted several important observations regarding both algorithmic behavior and usability. First, interactive graph visualization significantly improves the interpretability of network analyses, allowing users to better understand structural properties such as connectivity and clustering. Second, normalization steps, particularly the removal of isolated vertices, are essential for meaningful structural comparisons between graphs, especially in the context of isomorphism testing.

The project also demonstrated that classical graph algorithms, such as breadth-first search and connected component detection, provide powerful insights when applied to chemical space networks. Finally, the development process revealed that managing visualization state is a non-trivial aspect of interactive analysis tools and requires careful orchestration to ensure consistent results across different analyses.

Perspectives

With additional time and resources, several extensions could further enhance the software. Supporting labeled nodes and edges would allow a more explicit representation of chemical information, such as atom types and bond properties. The integration of chemoinformatics-specific file formats and reaction network analysis would broaden the scope of applications beyond static molecular similarity graphs.

Additional perspectives include the introduction of a command-line batch processing mode for large-scale analyses, as well as systematic performance benchmarking on larger datasets. Such developments would strengthen the applicability of the tool in research and educational contexts and provide a foundation for more advanced chemoinformatics workflows.

References

[1] Jeff Bezanson et al., *Julia: A Fresh Approach to Numerical Computing*, SIAM Review, 2017.

[2] LightGraphs.jl Development Team, *Graphs.jl Documentation*, https://juliagraphs.org